University of Toronto
Department of Mechanical and Industrial Engineering

Contest 1 Report

Where am I? Autonomous Robot Search of an Environment

MIE443: Mechatronics Systems - Design and Integration
Winter 2023

PRA0102
Thursday Group 7

Adam Cavallo 1004881979

Andres Cervera Rozo 1005469217

Selin Ertan 1004766096

Maggie Vuong 1005473961

Submitted: February 16th, 2023

Table of Contents

1.0 Problem Definition

The contest requires the TurtleBot to explore and map an unknown environment within 8 minutes autonomously, using sensory feedback to navigate. The robot cannot move faster than a speed limit to ensure mapping consistency. The environment has static objects, and the layout is unknown to the team in advance. Hence, the exploration algorithm must be robust to unknown environments with static obstacles. The functions, requirements, and the constraints of the contest are summarised in the table below.

Table 1: Functions, Requirements, Constraints of Contest 1

Functions	● Autonomously navigate ● Explore the scene ● Map the environment
Requirements	● Development of algorithm using C++ and ROS ● Use of robot sensors to help navigate the space
Constraints	● Time limit of 8 minutes ● No human intervention ● Speed limit of 0.25 m/s or 0.1m/s when close to walls/obstacles ● No use of move_base library

2.0 Strategy

To address the challenge, the team has chosen to implement a biased walk algorithm that proceeds forwards until a wall is detected directly in front. To decide on the direction that the robot should turn, a weighted decision is performed based on 3 independent factors. The first factor is that if given the option, the turtlebot would prefer to turn in a direction that has more open space to explore.

The second factor is a developing algorithm that favours moving away from the centroid of certain checkpoints on the map that have already been explored. At these distinct checkpoints, the robot will complete a full turn to analyse its surroundings. The locations of these checkpoints are all stored so that an informed decision can be made with regards to the general area that has already been explored thoroughly. This directly corresponds to the contest objective of exploring the unknown scene and it is the motivation for this particular factor. Figure 2.1 demonstrates a visualisation of this concept that was developed on a python notebook. By taking the dot product between vectors centroid→position, and position→leftwards, It can be determined whether the left direction is directed away from the centre.

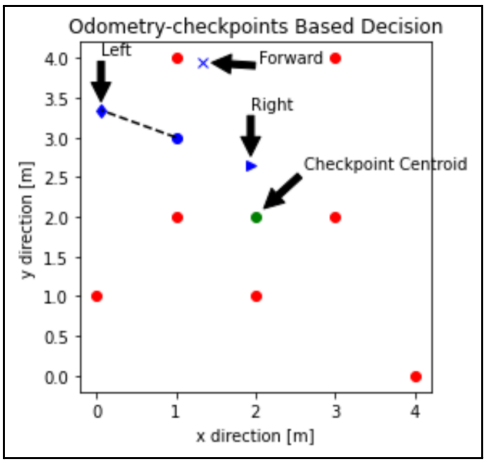

Figure 2.1: Example of Odometry Checkpoints. Checkpoints (red dots), centroid (green dot), current location (blue dot), forwards direction (blue x), left of robot (blue diamond), right of robot (blue triangle)

The final factor is responsible for determining if the area to the left or right has already been mapped on the Occupancy Grid. By taking a sample of the values of the cells in either direction, the turtlebot is able to distinguish if that area has an associated probability of occupancy. As seen in Figure 2.2, the values of the areas to the left and to the right are indexed according to the Yaw of the robot at that instance. This visualisation was also performed on a python notebook, and can be used to determine whether that area has been explored yet.

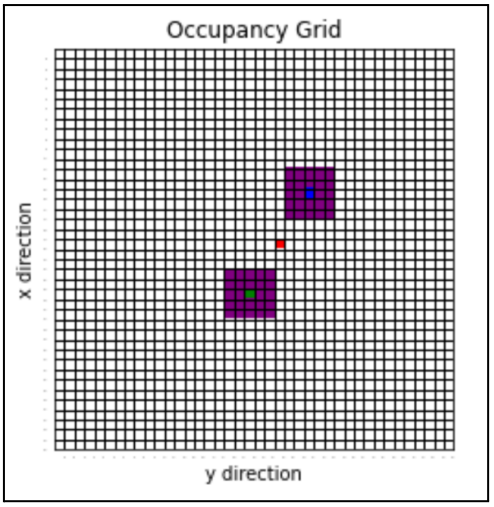

Figure 2.2: The location of the robot (red cell), with the left area (surrounding green cell), and the right area (surrounding blue cell)

By considering the bias direction of these 3 factors, and assigning a corresponding weight to the information from each function, a decision can be made on the most correct direction to turn towards. As the trial progresses, the weight of the checkpoint factor is increased so that further emphasis is put on moving away from explored areas.

From an array of 640 laser data points, it can be difficult to interpret what is a wall. To address this issue, the task is simplified by checking whether the distances at 5 distinct angles are above a certain threshold or not. These 5 locations are at the extreme ends of the laser line of sight (C locations), at the centre (A location), and at intermediate angles (B locations). The distance between the B locations (green diamonds) is slightly larger than the diameter of the turtlebot. By checking the distances at these 5 locations, the situation of the robot can be characterised into 6 distinct states, as described in detail in section 3.2.2. These 6 states allow the robot to adjust its rotational and linear velocities based on the environment, as well slow down when there are walls nearby. In order to calculate the specific threshold values for each distance, a python notebook was used to visualise the points, as seen in Figure 2.3. It should be noted that the distances for the C locations are visually farther because these points are used in the algorithm to check for nearby walls, and look for open areas.

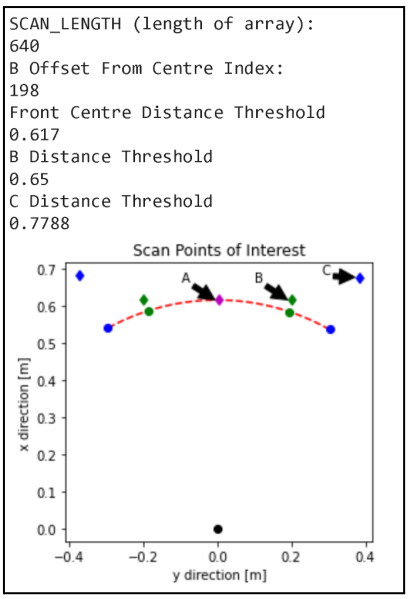

Figure 2.3: Output of Python Script used to determine values for Scan Message. A_thereshold or forwards direction (pink dot), B_threshold (green triangle), C_threshold (blue triangle)

Through this algorithm, the goal is that the robot keeps updating its priorities towards which direction is most favourable for the purposes of exploration. This tends to lead it to explore the perimeters, but the interior regions are accounted for by the periodic 360 rotations. As a safety measure, there are several running counters and parameters that are checking for whether the robot is stuck in a logic loop (such as infinite turns, or going in circles), and handles breaking that cycle to keep moving forward.

2.1 Modularity

Our team has decided to use GitHub in order to collaborate on our algorithm. We created individual branches to allow working on the code in parallel without interfering with each other's sections. The strategy was broken up into sub-functions such as forward, rotate, reverse, and determineScanType. This allowed the algorithm to be easily troubleshooted and navigated. In order to avoid merge issues, global variables and predetermined coding conventions were used.

3.0 Robot Design and Implementation

3.1 Sensory Design

The sensors used in the algorithm are summarised in the table below.

Table 3.1: Sensors Used in Algorithm

Sensors	Algorithm Implementation
Kinect Camera (Laserscan & Occupancy Grid message)	● Gather distance data from obstacles to quantify the environment ● Used for variables CMinus, BMinus, ACentre, BPlus, and CPlus (explained in Section 3). These variables are used in the function determineScanType() which detects environment orientation based on scan data (explained in Section 3.2.3). Based on the environment, the algorithm moves the robot (forward, reverse, or rotate).
Encoders (Odometry & Occupancy Grid messages)	● Gather robot's position data to detect environment and create checkpoints ● Used in functions distance() that returns the magnitude of change in distance between odom coordinates, and awayFromCentroid() that determines the direction the TurtleBot should move (functions explained in Section 3.2.3). Based on these functions, the robot keeps track of regions explored and moves away from those regions.
Bumper (Bumper messages)	● Detect obstacles below the scan plane ● Used in the function forward() to stop the robot from moving forward and activate the reverse() function instead (explained in Section 3.2.3)

3.2. Controller Design

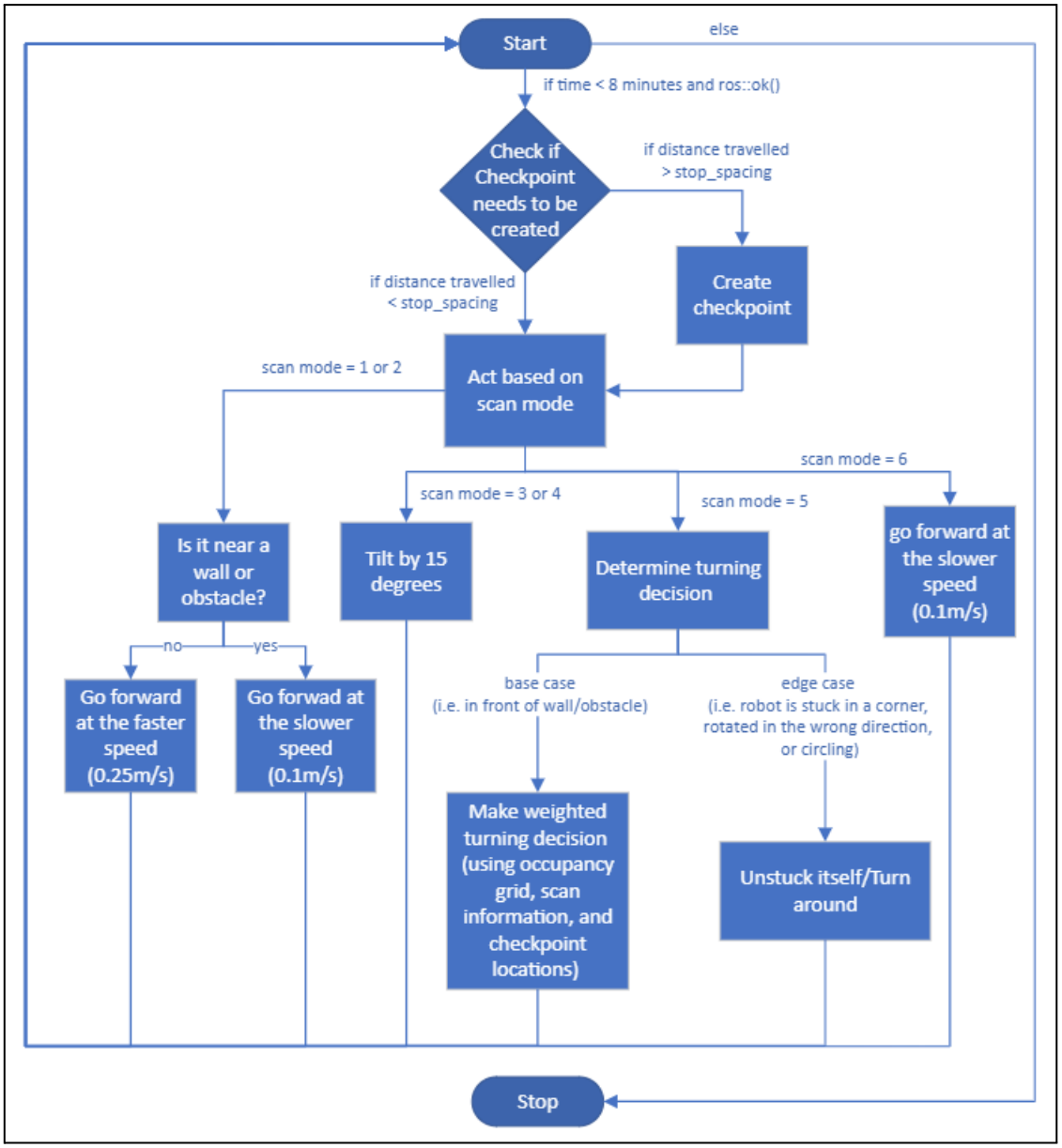

Figure 3.1: Flow Chart of Main Loop

Our algorithm uses a reactive control architecture which means that the robot does not try to plan and predict the future but rather it simply senses its environment and reacts [1]. The main loop or the robot's decision process is pictured in Figure 3.1. From this schematic, we can see that the Turtlebot is constantly checking the scan data in order to understand where it is in relation to the obstacles in front of it and acts on that information.

This was deemed to be sufficient as the Turtlebot's environment remains static throughout the contest. Furthermore, there is no reliable localization throughout the scene during the contest. This makes planning harder for the robot as it cannot plan its best path if it does not know the expected layout of the map. Therefore, from the problem definition, the robot would not need to be very smart as it does not need to learn or look far ahead to explore the environment. Its main concern is to steer clear of obstacles in its path and make its way around the scene.

3.2.1 Constants and Global Variables

Selected constants and global variables where the name of the variable may not have been sufficient in explaining their function have been given further explanations below. Omitted constants and global variables have been explained through comments found within the Contest Code in Appendix A.

ACentre

- This variable stores the range value (distance) found in the middle of the laser scan ranges message. In Figure 2.1, ACentre can be seen as the purple dot.

A/B/C/ACF_THRESHOLD

- These thresholds are representative of distances, measured in metres, that are in differences relative to the Kinect depth camera. Figure 2.1 visualises the thresholds as the diamond markers.

- A_THRESHOLD is the distance in front of the robot for which an obstacle would be considered in its way. Its default value is 0.617.
- B_THRESHOLD is the distance ahead used to find if the Turtlebot will fit through the passage. Its value is 0.65.
- C_THRESHOLD is the distance of the extremes of the scan array. Its value is set to 0.7788.
- ACF_THRESHOLD is the distance directly ahead of the robot and it is used to let the robot know when to slow down as it approaches an obstacle or wall. The default value is 0.9.

BMinus/Plus

- The minus indicates that the distance is to the right of the Turtlebot and plus would mean the opposite. The B indicates the distance of the passage that the robot would fit through. BMinus/Plus can be seen in Figure 2.1 as the green diamonds.

B_OFFSET

- It is the number of indices between point A and B in the scan array. Its default value is 198.

CMinus/Plus

- The minus indicates that the distance is to the right of the Turtlebot while the plus means the distance to the left. The C indicates the distance to the extremes of the laser scan. CMinus/Plus can be seen in Figure 2.1 as the blue diamonds.

CurrOdom

- This is a vector that has the approximate x and y coordinates ({x,y}) of the Turtlebot's current location based on its odometry.

L/LM/R/RM/M/LMR/LRState

- These variables were initialised with the corresponding bumper states array in the form of {(L)eft,(M)iddle,(R)ight}. A 1 would indicate that the bumper was pressed, while a 0 would indicate the opposite.
- For example, if the left bumper was activated, then then it would be shown through the LState array of {1,0,0}.

OCCUP/ODOM/SCAN_WEIGHT

- This is the weight given to each factor for when the Turtlebot makes its decision on where to go after a checkpoint.

- OCCUP_WEIGHT refers to the occupancy grid's weight in the decision. Its default is set to 1. The value of 1 was chosen because the team was not able to test on this with the physical robot, but still wanted to implement the logic that was developed so less weight was given.
- ODOM_WEIGHT refers to the weight assigned to odom_cmd in the main function which is the turning angle away from the centroid of checkpoints, which is the output of the awayFromCentroid function. Its weight is 20 but it should be noted that in the main function, ODOM_WEIGHT will be multiplied by some percentage as time progresses.
- SCAN_WEIGHT refers to the weight given to scan_cmd which is the clearest path. Its default value is 10.

SCAN_LENGTH

- This represents the total number of elements in the array from the scan message.
- It is set to 640. This value was found using the Turtlebot's scan message during testing and the following formula:

$$\frac{max\ angle - min\ angle}{angle\ increments} = total\ number\ of\ angles = length\ of\ scan\ array$$

STOPS_SPACING

- This is the distance, in metres, that must be travelled before the Turtlebot makes another checkpoint. Its value is set to 2.5m.

TurningBias

- This is either -1,0, or 1 from the C points as seen in Figure 2.3. A -1 means that the robot should turn to the right as it's an open area and 1 is the same but for the left side. A 0 indicates that either both are open or no preference for either side.

3.2.2 Callback Functions

bumperCallback

Input:	const kobuki_msgs::BumperEvent::ConstPtr& msg
Output:	N/A
Description:	This callback has two functions: 1. Saves all bumper states to the global variable, Bumper. 2. Checks if any bumper is pressed and saves AnyBumperPressed as true if pressed or false if not pressed.

laserCallback

Input:	const sensor_msgs::LaserScan::ConstPtr& msg
Output:	N/A
Description:	It assigns the following global variables from the laser scan ranges array: ● CMinus: rightmost non NaN value ● BMinus: value at minus B_OFFSET away from the centre ● ACentre: value at the centre ● BPlus: value at plus B_OFFSET away from the centre ● CPlus: leftmost value

occupancyCallback

Input:	const nav_msgs::OccupancyGrid::ConstPtr& msg
Output:	N/A
Description:	Dynamically updates the global variables that relate to the occupancy grid's dimensions and data (Map.data, MapResolution, MapWidth, and MapHeight) while also updating the global variables GridX and GridY that relay the Turtlebot's discrete position on the Occupancy Grid.

odomCallback

Input:	const nav_msgs::Odometry::ConstPtr& msg
Output:	N/A
Description:	It assigns the following global variables based on the odometry message. ● CurrOdom: x and y position to the corresponding index within the CurrOdom array ● Yaw: orientation of the robot that is converted from radians to degrees

3.2.3 User Defined Functions

arrayEqual

Input:	unit8_t array_1[2], unit8_t array_2[2]
Output:	boolean
Description:	Compares the elements of two arrays of the same length. Returns true if all values of both arrays are equal, otherwise, returns false. This function is used in *void reverse(ros::Publisher VelPub)* to compare the current bumper state with 7 arrays that represent each of the bumper state possibilities.

awayFromCentroid

Input:	N/A
Output:	Int -1,0,1int is_left_away_from_centroid
Description:	Calculates the centroid of the checkpoints and checks if the left direction is going away from the centre. A -1 corresponds to the right direction being the favourable direction, while 1 corresponds to the left direction, and 0 corresponds with an indifference to left/right directions. This is used in the main function to help the robot decide where to go after creating a checkpoint.

determineScanType

Input:	N/A
Output:	uint8_t state (1, 2, 3, 4, 5, or 6)
Description:	Based on A/B/C_THRESHOLD values, it uses the scan data in order to classify it into 1 of 6 options seen below. 1. Robot in open space, not near walls/obstacles 2. Object has been detected nearby, but not immediately blocking the robot 3. An object may be slightly blocking its path on the right side 4. An object may be slightly blocking its path on the left side 5. Approaching a wall or large obstacle, directly in front of the robot is blocked 6. Any other case This function is called in the main loop where its output is used to determine the robot's reaction to the current state.

distance

Input:	std::vector<float> prev_odom, std::vector<float> new_odom
Output:	float distance_magnitude_between_odom_inputs
Description:	Takes in the old and new locations of the robot. It finds the difference between the x and y coordinates and finds the distance between the two locations using Pythagoras. Returns the magnitude of the change in distance between the two odom inputs. This function is used in the main() function to find the distance travelled by the Turtlebot in order to decide if a new checkpoint is needed.

dotAndSign

Input:	float vector1[2], float vector2[2]
Output:	int sign(-1 or 1), int product
Description:	Takes in 2 vectors, each of size 2. Returns the dot product and its sign by calling the sign function. This function is used in the awayFromCentroid() function to find the distance and direction of the path furthest from the centre of the checkpoints.

forward

Input:	ros::Publisher VelPub, float linear_vel, float distance
Output:	N/A
Description:	Takes in desired velocity and desired distance to compute the time required to move forward at the desired velocity. Uses the time in order to move forward the desired distance. Also if a bumper is pressed while moving forward, then the robot is interrupted to stop, reverse, and rotate away from the obstacle.

mapValue

Input:	int width, int height
Output:	Map.data[position]
Description:	Takes in width and height coordinates and outputs the position on the Occupancy Grid Map. This is essential because the data of the Occupancy Grid is a 1 directional matrix. This function facilitates the access of a cell’s data through the width and height parameters.

	This function is used in the newFrontier function in order to localise the Turtlebot within the occupancy grid.

newFrontier

Input:	N/A
Output:	int (-1, 0, or 1)
Description:	Uses the occupancy grid to check the blocks/area directly to the right and left of the robot to provide a recommendation on the clearest path. Output is similar to awayFromCentroid. A -1 corresponds to the right direction being favoured while a 1 is left biassed, and 0 means that no direction is favoured. This is called in the main function to help the robot decide which direction to turn towards.

reverse

Input:	ros::Publisher VelPub
Output:	N/A
Description:	The reverse function serves the purpose of reversing and rotating the robot if the bumpers are pressed due to objects undetected by the scanner. The function uses arrayEqual to determine the current bumper state, and calls the *r*otate function with a correction angle as the input.

rotate

Input:	ros::Publisher VelPub, int angle_rot
Output:	N/A
Description:	This function is used to publish commands to rotate the robot based on a desired angle in degrees, inputted as int *angle_rot*

sign

Input:	float number
Output:	int 1 or -1
Description:	Returns 1 if the input number is positive and returns -1 if it is negative.

3.2.4 Main Function

The contest file's main function code can be found in Appendix A while a step through explanation can be seen below.

Main function - Step through explanation

Setup:

1. Node gets initialized as 'Navigator' with frequency of 10Hz
2. Subscribe to bumper, scan, odom, and map topics
3. Establish publisher to teleop topic
4. Initialize all counters and flags
5. Initialize vel message and start the timer
6. Initialize Odometry array as empty vector
7. Initialize Transform Listener

While under 8 mins:

(give a brief buffer period for incoming data to arrive successfully)

1. Listen to the transform between the map frame and the base_link frame to get an approximation of the position coordinates on the map
2. Get all the latest sensor data
3. Calculate how far we are from the last checkpoint
4. If we are far enough away:
    a. Do 360 rotation and `continue`
5. Scan the environments and determine scenario
6. If we can go forward, fast or slow?
7. If we need to tilt slightly, which direction? (B+ and B- used to determine tilt)
    a. Check if we are stuck in a loop of tilting repeatedly. If so, reverse and rotate
8. If there is a wall directly in front of us:
    a. Check if we just made a bad turn, or possibly 2 bad turns beforehand. If so, adjust accordingly
    b. Check if we've made too many turns in 1 direction recently. If so, turn around
    c. Check if we are stuck in a loop of turning repeatedly. If so, reverse and rotate
    d. Get clearest path option from determineScanType function
    e. Get far from checkpoints option from awayFromCentroid
    f. Get unexplored region option from newFrontier option
    g. Conduct a weighted decision matrix of all the options
    h. Rotate accordingly
9. If only A is blocked, move forward slowly as it could be noise in data
10. Update the timer
11. Delay for 100ms

If at any point there is a bumper triggered, it acts like an interrupt and immediately stops forward movement, reverses and rotates according to the triggered bumpers

4.0 Future Recommendations

To further improve our navigation algorithm, several different changes could be made. For instance, if our team had more time we would have considered the implementation of a reinforced learning algorithm, whereby the robot is able to learn from its incorrect turning decisions. This could be accomplished through the application of a punishment and reward system in our code that is able to change the decision weighting based on the robots previous incorrect and correct decisions. Another change we would make is to implement a numbering system for more specific recommendation weighting as opposed to the current system which outputs 1, 0 and -1. This would add a higher level of intelligence in the robot's decision making process, allowing multiple recommendations to have more influence on the robot's decisions. Lastly, with more time our team would have completed more test runs with the robot to properly isolate for different parameters, allowing us to find the optimal configuration in a more systematic way.

5.0 References

[1] G. Nejat, “MIE 443 LECTURE 4: Intelligent Control,” *Quercus*. [Online]. Available: https://q.utoronto.ca/courses/288207/files/folder/Contests/Contest%20%231?preview=24256737. [Accessed: 16-Feb-2023].

6.0 Contribution Table

Adam Cavallo	Constructed the reverse and rotate functions, troubleshooted, assisted with functions and future recs section of report and report editing
Andres Cervera Rozo	Developed algorithm logic, and the 3 functions used to determine a turning direction, wrote strategy section of report
Selin Ertan	Wrote the main code for bumperCallback, forward Tested and troubleshooted bumperCallback, forward, reverse, rotate, determineScanType, main Edited the report, wrote problem definition and sensory design section of report
Maggie Vuong	Wrote a random walk function (backup algorithm), created the skeleton (draft/ideas) of each section of the report, wrote the Controller design section and created a schematic for the algorithm, edited the report

Appendix A: Contest Code

```
// Advanced Main Algorithm
//MIE443 CONTEST 1

// Function variables: variable_name
// Functions: functionName
// Definitions: DEFINITION_NAME
// Global variables: VariableName

// 2 empty lines between functions
// 1 empty line after if, else if, else, for, and while (unless followed by another '}' )
// Initialization of variables at top in order of types
// Comments aligned on function level
// Comments start with space and capital

//CONVENTION
//CCW rotation is positive
//forward is x direction
//left is y direction

//!!!!!!!LIBRARIES!!!!!!!!!!!!!!!!!!!!
#include <ros/console.h>
#include "ros/ros.h"
#include <geometry_msgs/Twist.h>
#include <kobuki_msgs/BumperEvent.h>
#include <sensor_msgs/LaserScan.h>
#include <nav_msgs/Odometry.h>
#include <tf/transform_datatypes.h>
```

```
#include "std_msgs/String.h"
#include <nav_msgs/OccupancyGrid.h>
#include <tf/transform_listener.h>


#include <stdio.h>
#include <cmath>
#include <chrono>
#include <vector>
#include <math.h>

//!!!!!!CONSTANTS!!!!!!!!!!!!!!!!!!!!!
#define KOBUKI_DIAM 0.3515                      // Diameter of Turtlebot
#define N_BUMPER (3)                            // Number of bumpers on kobuki base
#define RAD2DEG(rad) ((rad) * 180. / M_PI)      // Coversion function
#define DEG2RAD(deg) ((deg) * M_PI / 180.)      // Inverse conversion function
#define SCAN_LENGTH 640                         // Elements in an array of scan, last index is SCAN_LENGTH-1
#define B_OFFSET 198                            // Indeces difference from A to either B
#define A_THRESHOLD 0.617                       // Considered obstacle directly in front
#define B_THRESHOLD 0.65                        // Considered wall in the window for passage
#define C_THRESHOLD 0.7788                      // Side (30 degree) distances to check for free space
#define ACF_THRESHOLD 0.9      //!!!!!!!          // Distance directly ahead to start slowing down by
#define ODOM_ARRAY_LENGTH 50                    // Max number of checkpoints per trial
#define FORWARD_FAST_V 0.25   //!!!!!            // Velocity when far from walls
#define FORWARD_SLOW_V 0.1   //!!!!!            // Velocity near walls
#define STEP_SIZE 0.15                          // Size of Forward steps (m)
#define BACKWARD_V -0.1                         // Magnitude and direction in x axis
#define BACKWARD_T 1.758                         // Move backwards a distance == to the radius of the turtlebot
#define LOOP_RATE 10                            // Rate for while loops that dictate callback and publish frequency (Hz)
#define TURNING_V 0.6         //!!!!!             // Rad/s
#define MAX_OBST_DIST 0.5                       // If less than this, we should turn, meters
#define STOPS_SPACING 2.5     //!!!!!           // Distance between checkpoints, meters
#define TURNING_LIM 500                         // Amounts of total degrees turned before turning robot around
#define OCCUP_WEIGHT 1                         // Weights for each factor
#define ODOM_WEIGHT 20
#define SCAN_WEIGHT 10
#define ADJUST_ANGLE 15                         // How much to adjust robot by when wall detected on sides, degrees
#define CONSECUTIVE_TILTS 8                     // Number of consecutive tilts to occur before thinking of it as a mistake
#define SIDE_DISTANCE 0.5           // Distance to check to our lefts for unexplored areas (m)
#define BLOCK_SIZE 2                // Area checked for occupancy is of size (BLOCK_SIZE * 2 + 1) ^2
#define TOTAL_BLOCKS (BLOCK_SIZE * 2 + 1) * (BLOCK_SIZE * 2 + 1)
#define UNEXPLORED_THRESH 0.4 * TOTAL_BLOCKS      // Max Threshold for whether a given area has been explored
// #define CHECKPOINT_RADIUS 0.2                // How close are we to a previous checkpoint, may remove
const int CENTRE_INDEX = SCAN_LENGTH / 2 -1;    // Get index for the middle index for the A value


//!!!!!!!!!!GLOBAL VARIABLES!!!!!!!!!!!!!!!!!!!!!!!!!


uint8_t LState[3] = {1,0,0};
uint8_t LMState[3] = {1,1,0};
uint8_t RState[3] = {0,0,1};
uint8_t RMState[3] = {0,1,1};
uint8_t MState[3] = {0,1,0};
```

```
uint8_t LMRState[3] = {1,1,1};
uint8_t LRState[3] = {1,0,1};

uint8_t PressedBumper[3]={0,0,0};               // 3 bit array of bumper status
std::vector<float> CurrOdom{0, 0};              // Where the turtlebot is in that moment
// std::vector<float> XboxRanges = {0};         // Storage for scan topic messages
std::vector<std::vector<float>> OdomArray{ODOM_ARRAY_LENGTH, std::vector<float>(2)}; // Full Array saving all checkpoint locations


long int MapWidth = 1; //Number of cells left to right
long int MapHeight = 1; // Number of cells bottom to top
uint8_t Bumper[3] = {kobuki_msgs::BumperEvent::RELEASED, kobuki_msgs::BumperEvent::RELEASED, kobuki_msgs::BumperEvent::RELEASED};
uint8_t LeftState = Bumper[kobuki_msgs::BumperEvent::LEFT];     // kobuki_msgs::BumperEvent::PRESSED if bumper is pressed, kobuki_msgs::BumperEvent::RELEASED otherwise
int32_t NLasers=0, DesiredNLasers=0, DesiredAngle=5;            // Laser parameters
float Angular = 0.0;                                            // Declare rotational vel (z) rad/s
float Linear = 0.0;                                             // Declare linear velocity (x) m/s
float PosX = 0.0, PosY = 0.0, Yaw = 0.0;                        // Initialize odom position
float MinLaserDist = std::numeric_limits<float>::infinity();    // Set minimum distance, volatile variable
  float CMinus = 0;
  float BMinus = 0;
  float ACentre = 0;
  float BPlus = 0;
  float CPlus = 0;
  float MapResolution = 0;
  float MapX = 0;
  float MapY = 0;
  bool IsNearWall = true;                           // Can we go fast or not
  bool AnyBumperPressed = false;                    // Reset variable to false
  int TurningBias = 0;                              // Finds clearest path or correction direction
  int GridX = 0;
  int GridY = 0;



  ros::Publisher VelPub;
  ros::Publisher OutputPub;
  geometry_msgs::Twist Vel;                                     // Create message for velocities as Twist type
  std_msgs::String Message;
  std::stringstream StringContent;
  nav_msgs::OccupancyGrid Map;


// !!!!!!!!!!!DECLARE FUNCTIONS!!!!!!!!!!!!!!!!!!!!!!!
void reverse(ros::Publisher VelPub);
void rotate(ros::Publisher VelPub, int angle);

//!!!!!!!!!!CALLBACK FUNCTIONS!!!!!!!!!!!!!!!!!!!!!!!
void bumperCallback(const kobuki_msgs::BumperEvent::ConstPtr& msg)
{
    // Access using bumper[kobuki_msgs::BumperEvent::{}] LEFT, CENTER, or RIGHT
    Bumper[msg->bumper] = msg->state;                           // Assigns bumper array to match message based on msg message
    AnyBumperPressed = false;
    for (uint8_t i = 0; i < N_BUMPER; i++){
        if (Bumper[i] == kobuki_msgs::BumperEvent::PRESSED){
            // bumper[0] = leftState, bumper[1] = centerState, bumper[2] = rightState
            AnyBumperPressed = true;
            PressedBumper[i] = 1;
        }

        else PressedBumper[i] =0;
    }

    if (AnyBumperPressed) reverse(VelPub);
}
```

```
148  void laserCallback(const sensor_msgs::LaserScan::ConstPtr& msg)
149  {
150      // ROS_INFO("ranges: %f", msg->ranges[0]);
151      CMinus = msg->ranges[7];
152      BMinus = msg->ranges[CENTRE_INDEX - B_OFFSET];
153      ACentre = msg->ranges[CENTRE_INDEX];
154      BPlus = msg->ranges[CENTRE_INDEX + B_OFFSET];
155      CPlus = msg->ranges[SCAN_LENGTH-1];
156  }
157
158
159  void odomCallback (const nav_msgs::Odometry::ConstPtr& msg)
160  {
161      CurrOdom[0] = msg->pose.pose.position.x;                // Get x position
162      CurrOdom[1] = msg->pose.pose.position.y;                // Get y position
163      Yaw = RAD2DEG(tf::getYaw(msg->pose.pose.orientation));  // Covert from quaternion to Yaw
164      // ROS_INFO("IN ODOM LOOP, x pose: %f",  msg->pose.pose.position.x);
165      // ROS_INFO("Position: (%f, %f) Orientation: %f rad or %f degrees.", PosX, PosY, Yaw, RAD2DEG(Yaw));
166  }
167
168
169   void occupancyCallback(const nav_msgs::OccupancyGrid::ConstPtr& msg)
170   {
171       // Map starting point on bottom left of map, 1D array 8 bit
172       Map.data = msg->data;
173       MapResolution =msg->info.resolution;
174       MapWidth = msg->info.width;
175       MapHeight = msg->info.height;
176
177
178       float origin_x = msg->info.origin.position.x;
179       float origin_y = msg->info.origin.position.y;
180
181       GridX = round((MapX - origin_x) / MapResolution);
182       GridY = round((MapY - origin_y) / MapResolution);
183   }
184
185
186   //!!!!!!!!!USER-DEFINED FUNCTIONS!!!!!!!!!!!!!!!!!!!!!!!!
187   bool arrayEqual(uint8_t array_1[], uint8_t array_2[]) {
188       for (int i =0; i< 2; i++){
189           if (array_1[i] != array_2[i] ) {
190               return false;
191           }
192       }
193
194       return true;
```

```
}


int sign(float number) {
    return (number>0)? 1: -1;                               // Positive -> 1, otherwise -1
}


int dotAndSign(float vector1[2], float vector2[2]) {
    float product = vector1[0]*vector2[0] + vector1[1]*vector2[1];
    return sign(product);                                   // Dot product and its sign
}


float distance(std::vector<float> prev_odom, std::vector<float> new_odom){     // Magnitude of change in distance between odom coordinates
    // ROS_INFO("new_x: %f, old_x: %f", new_odom[0], prev_odom[0]);
    float A = (new_odom[0] - prev_odom[0]);
    float B = (new_odom[1] - prev_odom[1]);
    float C_squared = A*A + B*B;
    // ROS_INFO("A: %f, B: %f, C_squared: %f", A, B, C_squared);
    return (std::sqrt(C_squared));
}


int forward(ros::Publisher VelPub, float linear_vel, float distance)
{
    // if (linear_vel == float(FORWARD_FAST_V)) ROS_INFO("forward fast!");
    // else if (linear_vel == float(FORWARD_SLOW_V)) ROS_INFO("forward slow");
    // else ROS_INFO("Forward but unsure why, linear_vel is: %f", linear_vel);
    ros::Rate loop_rate(LOOP_RATE);
    std::chrono::time_point<std::chrono::system_clock> start;       // Initialize timer

    Vel.angular.z = 0.0;
    Vel.linear.x = linear_vel;
    uint64_t secondsElapsed = 0;                                    // Variable for time that has passed
    float time_forward = abs(distance/linear_vel);

    start = std::chrono::system_clock::now();                       // Start the timer

    while (ros::ok() && secondsElapsed <= time_forward && !AnyBumperPressed) {
        VelPub.publish(Vel);
        ros::spinOnce();                                            // Listen to all subscriptions once
        // if (AnyBumperPressed) {
        //     break;
        // }
        loop_rate.sleep();
        secondsElapsed = std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-start).count(); //count how much time has passed
    }
}


void reverse(ros::Publisher VelPub) {                               // Called if any bumper pressed
    ROS_INFO("Object hit, reversing");
    ros::Rate loop_rate(LOOP_RATE);
    std::chrono::time_point<std::chrono::system_clock> start;       // Initialize timer

    Vel.linear.x = BACKWARD_V;                                      // Set linear to Twist
    Vel.angular.z = 0;
    uint64_t seconds_elapsed = 0;                                   // New variable for time that has passed
    int angle = 0;

    start = std::chrono::system_clock::now();                       // Start new timer at current time

    while (ros::ok() && seconds_elapsed <= BACKWARD_T) {            // Kobuki diameter is 0.3515m, we want to travel half (3.5s x 0.05m/s)
        VelPub.publish(Vel);                                        // Publish cmd_vel to teleop mux input
        loop_rate.sleep();
        seconds_elapsed = std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-start).count(); // Count how much time has passed
        //ROS_INFO("seconds_elapsed:%i,backward_T:%f",seconds_elapsed,BACKWARD_T);
    }

    if (arrayEqual(PressedBumper,LState)) angle = -45;            // Left bumper, rotate right (right is neg)

    else if (arrayEqual(PressedBumper, LMState)) angle = -90;      // Left and middle pressed only

    else if (arrayEqual(PressedBumper,RState)) angle = 45;        // Right pressed only, rotate left (pos)
```

```
270
271     else if (arrayEqual(PressedBumper,RMState)) angle = 90;       // Right and middle pressed only, rotate left
272
273     else if (arrayEqual(PressedBumper,MState)) {                  // Middle pressed only
274         // Rotate random angle
275         int array_angles[2] = {90, -90};                          // Array of random angles
276         int rand_num = rand() % 2;                                // Random number from 0 to 1
277         angle = array_angles[rand_num];                           // Index the array angles using random number, to rotate either left or right 90 degrees
278     }
279
280     else if  (arrayEqual(PressedBumper,LMRState) || arrayEqual(PressedBumper,LRState)) { // All pressed or left and right pressed
281        // Rotate random angle
282         int array_angles[2] = {135, -135};                        // Array of random angles
283         int rand_num = rand() % 2;                                // Random number from 0 to 1
284         angle = array_angles[rand_num];                           // Index the array angles using random number, to rotate either left or right 135 degrees
285     }
286
287     rotate(VelPub, angle);                                        // Rotate function
288 }
289
290
291 void rotate(ros::Publisher VelPub, int angle_rot){                // Called with input as an angle in degree
292     ROS_INFO("rotating by: %i",angle_rot);
293
294    int dir = sign(angle_rot);
295 //    ROS_INFO("direction: %i", dir);
296    float angle_rad = DEG2RAD(dir*angle_rot);                      // Function takes in an angle in degrees, converts it to rad
297    float turning_time = angle_rad/TURNING_V;                      // Calculates the time needed to turn based on constant turning speed, and the angle
298
299    std::chrono::time_point<std::chrono::system_clock> start;      // Initialize timer
300    start = std::chrono::system_clock::now();                      // Start new timer at current time
301    uint64_t seconds_elapsed = 0;                                  // New variable for time that has passed
302
303    ros::Rate loop_rate(100);
304    Vel.linear.x = 0;
305    Vel.angular.z = dir * TURNING_V;                               // Set turning speed
306 //    ROS_INFO("direction: %f", dir*TURNING_V);
307     while (ros::ok() && seconds_elapsed <= turning_time) {        // Turn until the turning time needed to complete the angle is passed
308         VelPub.publish(Vel);                                      // Start turning
309         loop_rate.sleep();
310         seconds_elapsed = std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-start).count(); //count how much time has passed
311     }
312 }
```

```
313
314
315 uint8_t determineScanType() { // Detect environment classification based on scan data
316     uint8_t state = 0;
317     IsNearWall = true;
318     TurningBias = 0;
319     bool CM = 0;
320     bool BM = 0;
321     bool AC = 0;
322     bool BP = 0;
323     bool CP = 0;
324
325     bool AC_far = 0;
326
327     if (CMinus > C_THRESHOLD) CM = 1;
328     if (BMinus > B_THRESHOLD) BM = 1;
329     if (ACentre > A_THRESHOLD) AC = 1;
330     if (BPlus > A_THRESHOLD) BP = 1;
331     if (CPlus > C_THRESHOLD) CP = 1;
332
333     if (ACentre > ACF_THRESHOLD) AC_far = 1;
334
335     // ROS_INFO("5 points left to right: %i, %i, %i, %i, %i", CP, BP, AC, BM, CM);
336
337
```

```
338       if (CM && BM && AC_far && BP && CP) {
339           state = 1;                      // Forward fast
340           IsNearWall = false;
341       }
342
343       else if (BM && AC && BP) {
344           state = 2;                      // Forward slow
345           if (!AC_far) ROS_INFO("far distance is blocked, going slower");
346           else if (!CM && !CP) ROS_INFO("C minus and plus are not clear");
347           else if (!CM) ROS_INFO("Just C minus blocked");
348           else if (!CP) ROS_INFO("Just C plus blocked");
349       }
350
351       else if (AC && BP) {
352           state = 3;
353           TurningBias = 1;                // Tilt left
354       }
355
356       else if (AC && BM) {
357           state = 4;
358           TurningBias = -1;               // Tilt right
359       }
360
361       else if ((!AC && !BP) || (!AC && !BM) || (!BM && !BP)) {
362           state = 5;
363           if (CP) {
364               TurningBias += 1;       // Turn left
365               ROS_INFO("Open space left");
366           }
367           if (CM) {
368               TurningBias = -1;       // Turn right
369               ROS_INFO("Open space right");
370           }
371       }
372
373       else {
374           state = 6;                      // A is neg or nan, or noise
375       }
376
377       return state;
378   }
379
380
381   int awayFromCentroid() {                        // Which direction will guide turtlebot away from centroid of checkpoints
382       float x_sum = 0;
383       float y_sum = 0;
384       float x_i = 0;
385       float y_i = 0;
386       uint8_t len = 1;
387
388       for (int i = 1; i++; i < ODOM_ARRAY_LENGTH) {
389           x_i = OdomArray[i][0];
390           y_i = OdomArray[i][1];
391           if (x_i == 0 && y_i == 0){
```

```
            break;
        }

        x_sum += x_i;
        y_sum += y_i;
        len += 1;
    }

    float x_centroid = x_sum / len;
    float y_centroid = y_sum / len;

    float difference[2] = {(CurrOdom[0] - x_centroid), (CurrOdom[1] - y_centroid)};

    float yaw_rad_left = DEG2RAD(Yaw + 180);
    float unit_vector_left[2] = {cosf(yaw_rad_left), sinf(yaw_rad_left)};
    int is_left_away_from_centroid = dotAndSign(unit_vector_left, difference);
    return is_left_away_from_centroid;
}


int mapValue(int width, int height) {
    int position = MapWidth * height + width;
    return Map.data[position];
}


 int newFrontier() {
     long int surrounding_x = 0;
     long int surrounding_y = 0;
     float left_counter = 0;
     float right_counter = 0;
     bool left_in_bounds = true;
     bool right_in_bounds = true;
     int occup_dec = 0;
     //GridX and GridY are zero based

     float x_delta = SIDE_DISTANCE * sinf(Yaw);
     float y_delta = SIDE_DISTANCE * cosf(Yaw);
     int x_disc = round(x_delta/MapResolution);
     int y_disc = round(y_delta/MapResolution);


     long int left_index_x = GridX - x_disc;
     long int left_index_y = GridY - y_disc;
     long int right_index_x = GridX + x_disc;
     long int right_index_y = GridY + y_disc;

     long int left_centre[2] = {0, 0};
     long int right_centre[2] = {0, 0};
```

```
445     if ((left_index_x < 0) || (left_index_y < 0) || (left_index_x >= MapWidth) || (left_index_y >= MapHeight)) {
446         //Left is out of map bounds
447         left_in_bounds = false;
448     }
449 
450     else {
451         left_centre[0] = left_index_x;
452         left_centre[1] = left_index_y;
453     }
454 
455     if ((right_index_x < 0) || (right_index_y < 0) || (right_index_x >= MapWidth) || (right_index_y >= MapHeight)) {
456         //Right is out of map bounds
457         right_in_bounds = false;
458     }
459 
460     else {
461         right_centre[0] = right_index_x;
462         right_centre[1] = right_index_y;
463     }
464 
465 
466     for (int i = -BLOCK_SIZE; i <= BLOCK_SIZE; i++) {
467         for (int j = -BLOCK_SIZE; j <= BLOCK_SIZE; j++) {
468             // surrounding_in_bounds = true;
469             if (left_in_bounds) {
470                 surrounding_x = left_centre[0] + i;
471               surrounding_y = left_centre[1] + j;
472 
473               if ((surrounding_x >= 0) && (surrounding_y >= 0) && (surrounding_x < MapWidth) && (surrounding_y < MapHeight)) {
474                   // If the dot in the left surounding area is a valid point in map
475                   // left_counter = left_counter + occup_map_array[surrounding_x, surrounding_y] mapValue
476                   if (mapValue(surrounding_x, surrounding_y) < 0) {
477                       left_counter = left_counter + 1;
478                   }
479               }
480           }
481 
482           if (right_in_bounds) {
483               surrounding_x = right_centre[0] + i;
484               surrounding_y = right_centre[1] + j;
485 
486               if ((surrounding_x >= 0) && (surrounding_y >= 0) && (surrounding_x < MapWidth) && (surrounding_y < MapHeight)) {
487                   // If the dot in the right surounding area is a valid point in map
488                   if (mapValue(surrounding_x, surrounding_y) < 0) {
489                       right_counter = right_counter + 1;
490                   }
491               }
492           }
493       }
494   }
```

```
    if (left_counter > UNEXPLORED_THRESH) occup_dec += 1;
    if (right_counter > UNEXPLORED_THRESH) occup_dec -= 1;
    return occup_dec;
}


// !!!!!!!!!!!!!MAIN!!!!!!!!!!!!!!!!!!!!!!!!!!
int main(int argc, char **argv)
{
    ros::init(argc, argv, "Navigator");                    // Node name
    ros::NodeHandle nh;                                         // Iinitialize node handler
    ros::Rate loop_rate(LOOP_RATE);                             // Code will try to run at LOOP_RATE Hz

    ros::Subscriber bumper_sub = nh.subscribe("mobile_base/events/bumper", 10, &bumperCallback); //subscribe to bump topic with callback
    ros::Subscriber laser_sub = nh.subscribe("scan", 10, &laserCallback);       // Subscribe  to /scan topic
    ros::Subscriber odom_sub = nh.subscribe("odom", 1, &odomCallback);
    ros::Subscriber map_sub = nh.subscribe("map", 1, &occupancyCallback);
    VelPub = nh.advertise<geometry_msgs::Twist>("cmd_vel_mux/input/teleop", 1); // Publish cmd_velocity as Twist message
    OutputPub = nh.advertise<std_msgs::String>("feedback", 1000);
    uint64_t seconds_elapsed = 0;                               // Variable for time that has passed
    uint8_t scan_mode = 0;                                      // List of X states
    uint8_t bad_tilt_counter = 0;                               // Number of consecutive tilts done
    uint8_t bad_turn_counter = 0;
    int checkpoint_counter = 1;
    float rotation = 0;
    float dist_travelled = 0;                                   // How much moved since last checkpoint
    float percent_run = 0;
    long turning_count = 0;                                     // Accumulation of how much travelled
    bool just_rotated = false;                                  // Did we just make a 90 degree turn?
    bool is_in_corner = false;                                  // Did we just make a 90 degree turn?
    int decision = 0;
    int scan_cmd = 0;
    int occup_cmd = 0;
    int odom_cmd = 0;
    long int seconds_long = 0;
    long int loop_iterations = 0;
```

```
531
532
533
534         ros::spinOnce();
535         std::vector<float> last_odom = CurrOdom;                          // Location of last point
536         OdomArray[0] = last_odom;
537
538         geometry_msgs::Twist vel;                                         // Create message for velocities as Twist type
539         std::chrono::time_point<std::chrono::system_clock> start;         // Initialize timer
540         start = std::chrono::system_clock::now();                         // Start the timer
541         tf::TransformListener listener;
542
543
544         while(ros::ok() && seconds_elapsed <= 480) {        // While ros master running and < 8 minutes
545             tf::StampedTransform transform;
546             try {
547                 listener.lookupTransform("/map", "/base_link", ros::Time(0), transform);
548             }
549             catch (tf::TransformException &ex) {
550                 ROS_ERROR("%s",ex.what());
551                 ros::Duration(1.0).sleep();
552             }
553
554             MapX = transform.getOrigin().x();
555             MapY = transform.getOrigin().y();
556             // ROS_INFO("X: %f, Y %f", MapX, MapY);
557
558
559             ros::spinOnce();                                // Listen to all subscriptions once
560             loop_iterations += 1;
561             if (loop_iterations < 5) {
562                 ros::spinOnce();
563                 loop_rate.sleep();
564                 continue;
565
566             }
567
568
```

```
600         seconds_long = seconds_elapsed;
601         ROS_INFO("Seconds elapsed: %li", seconds_long);
602         percent_run = ((float)seconds_long / 480.0) + 0.01; //How much of the trial time has passed, but we want non-zero in all cases
603         ROS_INFO("percent_run: %f", percent_run);
604         ROS_INFO("\nTime: %ld, Pos: (%f, %f),  Ori: %f deg, turning_cnt: %ld", seconds_long, CurrOdom[0], CurrOdom[1], Yaw, turning_count); //pr
605         dist_travelled = distance(last_odom, CurrOdom); // Distance is function to determine distance between 2 coordinates
606         //TO-DO!!!!!!!!!!
607         //add criteria to do checkpoint of forward happens uninterreupted
608         //TO-DO!!!!!!!!!!
609
610         if (dist_travelled > STOPS_SPACING) {           // Checks space since last checkpoint
611             ROS_INFO("New checkpoint location, doing 360");
612             last_odom = CurrOdom;                       // Reset reference location
613             dist_travelled = 0;
614             OdomArray[checkpoint_counter] = CurrOdom;   // Keeps track of locations of all past checkpoints, leaves first index as 0,0
615             checkpoint_counter += 1;
616             // rotate(VelPub, 360);                        // Scan area. positive is counterclockwise
617             rotate(VelPub, 90);
618             rotate(VelPub, 90);
619             rotate(VelPub, 90);
620             rotate(VelPub, 105);
621             // rotate(VelPub, 600); // Just for simulation
622             continue;
623         }
624
625         scan_mode = determineScanType();                // Recognize different possibilities based on scan array, also look at changes in odom
626         ROS_INFO("Scan Mode: %i", scan_mode);
627
628         // !!!!!Print message to topic: feedback
629         StringContent << "Scan Mode: " << scan_mode;
630         Message.data = StringContent.str();
631         OutputPub.publish(Message);
632
633         if (scan_mode < 3) {
634             if (IsNearWall) {
635                 ROS_INFO("Forward slow");
636                 forward(VelPub, FORWARD_SLOW_V, STEP_SIZE);
637             }
```

```
638
639             else {
640                 ROS_INFO("Forward Fast");
641                 forward(VelPub, FORWARD_FAST_V, STEP_SIZE);
642
643             }
644             // IsNearWall? forward(VelPub, FORWARD_SLOW_V, STEP_SIZE): forward(VelPub, FORWARD_FAST_V, STEP_SIZE);
645
646
647             bad_tilt_counter = 0;
648             bad_turn_counter = 0;
649             just_rotated = false;
650             is_in_corner = false;
651             loop_rate.sleep();
652         }
653
654         else if ((scan_mode == 3) || (scan_mode == 4)){ //Tilt
655             ROS_INFO("Minor tilt");
656             just_rotated = false;
657             is_in_corner = false;
658             bad_turn_counter = 0;
659             if (bad_tilt_counter >= CONSECUTIVE_TILTS) {
660                 reverse(VelPub);
661                 rotate(VelPub, 180);
662                 ROS_INFO("I'm stuck in a tilt loop, turning around now");
663                 turning_count = 0;
664                 bad_tilt_counter = 0;
665                 continue;
666             }
667
668             rotation = TurningBias * ADJUST_ANGLE;
669             rotate(VelPub, rotation);                   // Fix allignment by ~ 15 degrees depending on direction of bias
670             turning_count += rotation;                  // Keep track of turning
671             bad_tilt_counter += 1;                      // Track how many times we've done a consecutive tilt
672         }
673
674         else if (scan_mode == 5) {                      // Front wall
675             if (bad_turn_counter >= CONSECUTIVE_TILTS) {
676                 reverse(VelPub);
677                 rotate(VelPub, 180);
678                 ROS_INFO("I'm stuck in a turn loop, turning around now");
679                 turning_count = 0;
680                 bad_turn_counter = 0;
681                 continue;
682             }
683             bad_tilt_counter = 0;
684             bad_turn_counter += 1;
685             if (just_rotated) {                         // Bad turning decision, we're at a corner and turned into other wall
686                 ROS_INFO("I think I'm in a corner and going the other way");
687                 rotate(VelPub, 180);                    // Turn around, think about a dead end corridor situation
688                 is_in_corner = true;
689                 turning_count += -decision*180;         // If we made the wrong choice before, we want to overwrite that contribution to the count and add accordingly
690                 just_rotated = false;                   // Reset flag
691
692                 continue;
693             }
694
695             else if (is_in_corner) {
696                 ROS_INFO("I think I just hit a 3 way dead end and am going back");
697                 rotate(VelPub, -decision*90);
698                 turning_count += decision*90;
699                 is_in_corner = false;
700                 continue;
701             }
702
703             if (turning_count > TURNING_LIM) {          // We've been turning too much
704                 ROS_INFO("I think I rotated too much left, turning around");
705                 rotate(VelPub, 180);                    // Go other direction
706                 turning_count = 0;                      // Reset flag
707                 continue;
708             }
709
```

```
            else if (turning_count < -TURNING_LIM) {    // Same as above in other direction
                ROS_INFO("I think I rotated too much right, turning around");
                rotate(VelPub, 180);
                turning_count = 0;
                continue;
            }

            scan_cmd = TurningBias;                     // Based on clearest path, L or R, function
            odom_cmd = awayFromCentroid();             // Based on nearby odom checkpoints, function, checks OdomArray
            occup_cmd = newFrontier();                  // Based on nearby occupancy grid, function



            decision = sign((float)scan_cmd*(float)SCAN_WEIGHT + (float)odom_cmd*(float)ODOM_WEIGHT*percent_run + (float)occup_cmd*(float)OCCUP_WEIGHT); // Weighted decision
            ROS_INFO("Scan: %i, Odom: %i,  Occupancy: %i, Decision: %i", scan_cmd, odom_cmd, occup_cmd, decision);
            //TO-DO!!!!!!!!!
            //rather than weight, potentially do steps to see if that area's been explored
            //TO-DO!!!!!!!!!
            rotation = 90*decision;
            rotate(VelPub, rotation);                   // Rotate 90 based on decision
            turning_count += rotation;                  // Add rotation to count
            just_rotated = true;
        }

        else {
            // rotate(VelPub, 180); //robot is confused
            forward(VelPub, FORWARD_SLOW_V, STEP_SIZE);
            turning_count = 0;
            ROS_INFO("State 6, just going forward slowly");
            bad_tilt_counter = 0;
            bad_turn_counter = 0;
        }

        seconds_elapsed = std::chrono::duration_cast<std::chrono::seconds>(std::chrono::system_clock::now()-start).count(); //count how much time has passed
        loop_rate.sleep();                              // Delay function for 100ms
    }
    // delete MapPointer;
    return 0;
}
```

Appendix B: Python Notebooks Code

A/B/C Thresholds

```
import math
import numpy as np
from matplotlib import pyplot as plt
min_angle = -0.513185441494
max_angle = 0.49990695715
angle_increment = 0.0015854307451

d_centre2cam = 0.09 #distance from centreof robot to front of camera
min_distance = 0.55 #robot cannot read less
clearence_width = 0.4 # space w need in order to go straight
phi = 0.35 #Forward distance for turning
C_mult = 1.1
```

```
#@title
ints = round((max_angle-min_angle)/angle_increment)
print(ints)


turtlebot_r = 0.354 #radius of bot
hidden_dist = turtlebot_r/2 + d_centre2cam # distance taken up by bot footprint
front_blind = min_distance - (hidden_dist) # space in front we are blind to
print(front_blind)



half_width = clearence_width/2 #for trig

forward_boundary_angle = math.atan(half_width/(hidden_dist+phi))
forward_boundary_angle_deg = forward_boundary_angle*180/math.pi

print(forward_boundary_angle_deg)

#beta = forward_boundary_angle_deg
```

```
#@title
offset = math.pi/2

total_ints = ints +1
print("SCAN_LENGTH (length of array): ")
print(total_ints)
angles = np.linspace(offset + min_angle, offset + max_angle, num = (total_ints))

factor = hidden_dist + phi
sin_angles = np.sin(angles)
cos_angles = np.cos(angles)




B_offset = round(forward_boundary_angle/angle_increment)
print("B Offset From Centre Index: ")
print(B_offset)

A_index = round(total_ints/2) # +- 5
B_plus = A_index + B_offset # 5-
B_minus = A_index - B_offset #5+
C_minus = 0 #5+
C_plus = total_ints-1   #5-

print("Front Centre Distance Threshold")
print(factor)
B_dist = round(factor / sin_angles[B_minus], 3)
print("B Distance Threshold")

print(B_dist)


C_dist = round(factor / sin_angles[C_minus], 3) * C_mult
print("C Distance Threshold")
print(C_dist)

C_Factor = C_dist / factor
```

```
sin_angles *= factor
cos_angles *= factor
fig, axs = plt.subplots(1, 1)
plt.title("Scan Points of Interest")
plt.xlabel("y direction [m]")
plt.ylabel("x direction [m]")
plt.plot(cos_angles,sin_angles, '--r')

plt.plot(cos_angles[C_minus],sin_angles[C_minus], 'ob')
plt.plot(cos_angles[B_minus],sin_angles[B_minus], 'og')
plt.plot(cos_angles[A_index],sin_angles[A_index], 'dm')
plt.plot(cos_angles[B_plus],sin_angles[B_plus], 'og')
plt.plot(cos_angles[C_plus],sin_angles[C_plus], 'ob')


# plt.plot(0,factor, 'ok')
plt.plot(-half_width, factor, 'dg')
plt.plot(half_width, factor, 'dg')
plt.plot(cos_angles[C_minus]*C_Factor,sin_angles[C_minus]* C_Factor, 'db')
plt.plot(cos_angles[C_plus]*C_Factor,sin_angles[C_plus]* C_Factor, 'db')


plt.plot(0,0, 'ok')
axs.set_aspect('equal', 'box')

axs.annotate('A', xy=(cos_angles[A_index]-0.01,sin_angles[A_index]), xytext=(cos_angles[A_index]-0.1,sin_angles[A_index]+0.05),
            arrowprops=dict(facecolor='black', shrink=0.01))
axs.annotate('B', xy=(half_width-0.01,factor), xytext=(half_width-0.1,factor+0.05),
            arrowprops=dict(facecolor='black', shrink=0.01))
axs.annotate('C', xy=(cos_angles[C_minus]*C_Factor-0.01,sin_angles[C_minus]*C_Factor), xytext=(cos_angles[C_minus]*C_Factor-0.1,sin_angles[C_minus]*C_Factor+0.01),
            arrowprops=dict(facecolor='black', shrink=0.01));
```

Checkpoint Centroid

```
import numpy as np
from matplotlib import pyplot as plt
import math


#!!!!!!!!!!!!!!!
#Red is all the checkpoints
#Green is the centroid
#Blue circle is where we are
#Blue X indicates the direction we are facing
#Blue diamond is the left direction
#Blue triangle is the right direction
#Dotted line is the direction algorithm chooses is better
odom_array = np.array([[0,1], [1,2], [3, 2], [2, 1], [4, 0], [1, 4], [3, 4]])

x = odom_array[:, 0]
y = odom_array[:, 1]

current_x = x_coordinate
current_y = y_coordinate

_len = len(x)
centroid_x = sum(x)/_len
centroid_y = sum(y)/_len

diff = np.array([(current_x-centroid_x), (current_y-centroid_y)])


angle = yaw
angle_rad = angle *math.pi / 180 + (math.pi/2)

left_angle = angle_rad + math.pi/2
right_angle = angle_rad - math.pi/2

unit_vector = np.array([math.cos(angle_rad), math.sin(angle_rad)])

unit_vector_left = np.array([math.cos(left_angle), math.sin(left_angle)])
unit_vector_right = np.array([math.cos(right_angle), math.sin(right_angle)])

result = np.sign(np.dot(unit_vector_left, diff))
print(result)
```

```
if (result == 1):
  left = True
else:
  left = False

fig, axs = plt.subplots(1, 1)
plt.title("Odometry-checkpoints Based Decision")
plt.xlabel("x direction [m]")
plt.ylabel("y direction [m]")
plt.plot(x,y, 'ro')
plt.plot(current_x, current_y, 'bo')

plt.plot(current_x+unit_vector[0], current_y+unit_vector[1], 'bx')


plt.plot(centroid_x, centroid_y, 'go')

plt.plot(current_x+unit_vector_left[0], current_y+unit_vector_left[1], 'bd')
plt.plot(current_x+unit_vector_right[0], current_y+unit_vector_right[1], 'b>')

if left:
  plt.plot([current_x, current_x+unit_vector_left[0]], [current_y, current_y+unit_vector_left[1]], 'k--')

else:
  plt.plot([current_x, current_x+unit_vector_right[0]], [current_y, current_y+unit_vector_right[1]], 'k--')

axs.set_aspect('equal', 'box')

axs.annotate('Left', xy=(current_x+unit_vector_left[0], current_y+unit_vector_left[1]+ 0.1), xytext=(current_x+unit_vector_left[0], current_y+unit_vector_left[1]+0.7),
            arrowprops=dict(facecolor='black', shrink=0.01))
axs.annotate('Right', xy=(current_x+unit_vector_right[0], current_y+unit_vector_right[1]+ 0.1), xytext=(current_x+unit_vector_right[0], current_y+unit_vector_right[1]+0.7),
            arrowprops=dict(facecolor='black', shrink=0.01))
axs.annotate('Forward', xy=(current_x+unit_vector[0]+ 0.1, current_y+unit_vector[1]), xytext=(current_x+unit_vector[0] + 0.7, current_y+unit_vector[1]),
            arrowprops=dict(facecolor='black', shrink=0.01))

axs.annotate('Checkpoint Centroid', xy=(centroid_x+ 0.1, centroid_y+ 0.1), xytext=(centroid_x + 0.6, centroid_y + 0.6),
            arrowprops=dict(facecolor='black', shrink=0.01))
```

Occupancy Grid

```
import math
import numpy as np
from matplotlib import pyplot as plt
from matplotlib import colors


grid_x = 19 # Forward
grid_y = 22 # Left


yaw = 60
side_distance = 0.3

map_width = 40
map_height = 40
resolution = 0.05

matrix_x = grid_x
matrix_y = grid_y

yaw_ros = yaw + 90
yaw_rad = yaw *math.pi/180

x_delta = side_distance * math.sin(yaw_rad)
x_disc = round(x_delta/resolution)

y_delta = side_distance * math.cos(yaw_rad)
y_disc = round(y_delta/resolution)

data = np.zeros(map_width * map_height).reshape(map_width, map_height)

EMPTY_CELL = 0
CURR_CELL = 1
LEFT_CELL = 2
RIGHT_CELL = 3
SURROUNDING_CELL = 4



data [matrix_x, matrix_y] = CURR_CELL

left_centre = np.array([matrix_x + x_disc, matrix_y - y_disc])
right_centre = np.array([matrix_x - x_disc, matrix_y + y_disc])


data [left_centre[0], left_centre[1]] = LEFT_CELL
data [right_centre[0], right_centre[1]] = RIGHT_CELL
```

```
block_size = 2
total_blocks = 0

left_counter = 0
right_counter = 0
for i in range(-block_size, block_size+1):
  for j in range(-block_size, block_size+1):
    left_counter += data [left_centre[0] + i, left_centre[1] + j]
    right_counter += data [right_centre[0] + i, right_centre[1] + j]
    data [left_centre[0] + i, left_centre[1] + j] = SURROUNDING_CELL
    data [right_centre[0] + i, right_centre[1] + j] = SURROUNDING_CELL
    total_blocks += 1

data [left_centre[0], left_centre[1]] = LEFT_CELL
data [right_centre[0], right_centre[1]] = RIGHT_CELL



left_avg = left_counter/total_blocks
right_avg = right_counter/total_blocks

print("total block")
print(total_blocks)


print("left avg")
print(left_avg)
print("right avg")
print(right_avg)







cmap = colors.ListedColormap(['white', 'red', 'green', 'blue', 'purple'])
bounds = [EMPTY_CELL, CURR_CELL, LEFT_CELL, RIGHT_CELL, SURROUNDING_CELL, SURROUNDING_CELL +1]
norm = colors.BoundaryNorm(bounds, cmap.N)

fig, ax = plt.subplots()
ax.imshow(data, cmap=cmap, norm=norm)
# draw gridlines
ax.grid(which='major', axis='both', linestyle='-', color='k', linewidth=1)
ax.set_xticks(np.arange(0.5, map_width, 1));
ax.set_yticks(np.arange(0.5, map_height, 1));
plt.tick_params(axis='both', labelsize=0, length = 0)
plt.title("Occupancy Grid")
plt.xlabel("y direction")
plt.ylabel("x direction")
plt.show()
```